Competitive Programming and Contests
Hands-On 1 – Report

Gabriele Marsili

October 2026

The solution is in `lib.rs`: I added two public methods to the provided `Tree`, `is_bst` and `sum_max_simple_path`, and a set of tests. In the following, n is the number of nodes and h the height of the tree.

Exercise 1 – Checking whether the tree is a BST

Considering the definition of a BST, it is necessary to decide a convention for keys with the same value, otherwise it is not clear how duplicates are handled: I decided to allow duplicates in the right subtree of a node.

An interesting observation is that comparing a node only with its children is not enough, because the subtrees rooted at the children could contain keys in the wrong order. For this reason I introduced an interval of admissible values for the key, passed down in the recursion (line 100):

```
fn rec_is_bst(&self, node_id: Option<usize>, admissible_interval: (u32, u32)) -> bool
```

This handles cases like the test `grandchild_violating_ancestor_is_rejected`: with root 10, left child 5 and 15 as the right child of 5, every parent-child pair is ordered, but 15 is in the left subtree of 10.

The visit is recursive. If the id of a node is `None` the node does not exist and I return `true` (base case). Otherwise I first check that the key is in the admissible interval and then I make the recursive calls on the subtrees (inductive cases) with the intervals $[low, key - 1]$ for the left one and $[key, high]$ for the right one. At the root the interval is $[\texttt{u32::MIN}, \texttt{u32::MAX}]$.

Observation: keys are `u32`, so $key - 1$ would underflow when the key is 0. I use `checked_sub` (line 114), which returns `None` in this case: since no `u32` key is smaller than 0, the left subtree must be empty.

Complexity:

- $O(n)$ time, since each node is visited at most once;
- $O(h)$ extra space for the recursion stack, where h is the height of the tree: $h = \log n$ if the tree is balanced, $h = n$ in the worst case (a chain).

Exercise 2 – Maximum Path Sum

The method returns the sum of the maximum simple path connecting two leaves, where a leaf is a node with no children. If the tree has fewer than two leaves no such path exists and the method returns `None`.

Observation: instead of using 0 for this case I return `None`, because 0 can also be the sum of a real path (for example two leaves and all keys equal to 0, test `zero_keys_are_distinguished_from_no_path`), so the two cases would not be distinguishable.

The method keeps a global maximum, of type `Option<u32>` and initially `None`, passed by mutable reference to a recursive depth-first visit (`dfs`, line 128) that starts from the root (id 0). If the node does not exist, `dfs` returns 0, since we are computing sums. Otherwise it computes recursively the values of the left and right subtrees (`l_max`, `r_max`) and the local sum $key + l_max + r_max$.

The local sum is a candidate only if the current node has both children (line 141): a path that bends at this node goes down to a leaf in the left subtree and to a leaf in the right subtree. With only one child it would end at the current node, which is not a leaf: see the test `path_cannot_end_at_a_node_with_one_child`, where the correct answer is 3 and updating at the root, which has one child, would give 1002. When the node has both children I update the global maximum: if it is still `None` it becomes the local sum, otherwise the maximum between the two.

In any case `dfs` returns $key + \max(l_max, r_max)$. This is the invariant of the visit: the returned value is the maximum sum of a path that starts at the node and goes down to a leaf of its subtree. For a leaf, both subtrees are empty and the value is just its key. When a node has only one child, the missing side is worth 0 and the other one is at least 0, since keys are `u32`, so the maximum always continues towards the existing child.

Complexity:

- $O(n)$ time, since each node is visited once with constant work;
- $O(h)$ extra space for the recursion stack, in addition to the tree itself.

Tests

All 19 tests pass, the code is formatted with `cargo fmt` and `cargo clippy` gives no warnings.

- `is_bst`: single node, a valid BST (the tree of the example in the statement), a wrong direct child on both sides, a grandchild that violates an ancestor, duplicates on the right (accepted) and on the left (rejected), extreme keys 0 and `u32::MAX`, key 0 with a left child, a chain of 10 000 nodes.
- `sum_max_simple_path`: single node and chain (`None`), root with two leaves, a node with only one child (not an endpoint, and traversed down to the leaf), best path not through the root, all keys equal to 0 (`Some(0)`), the tree of the statement.
- Randomized tests: both methods are compared with brute-force versions on 20 000 random trees each (an $O(n^2)$ check of the definition for `is_bst`, and for the path all the pairs of leaves through their lowest common ancestor).

Limitations

- Sums are `u32`, as in the provided `sum`: a path whose sum is larger than `u32::MAX` overflows (panic in debug builds).
- Both methods are recursive, so on very deep trees the recursion could exceed the stack size.

Use of AI

I used Claude Code to generate the tests, to review my code, and to translate into English and format this report.